

GIT

Familiar with Git commands like git init, git status, git add, git commit, git push, and git pull.

In this hands-on lab, you will learn how to

- Setup your machine with Git Configuration
- Integrate notepad++.exe to Git and make it a default editor
- Add a file to source code repository

Prerequisites

- Install Git Bash client in your machine

Notes*:

Please follow the below steps for creating a free account in GitHub.
Don't use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes.**

Step 1: Setup your machine with Git Configuration

To create a new repository, signup with GitLab and register your credentials

Login to GitLab and create a "GitDemo" project

1. To check if Git client is installed properly: Open Git bash shell and execute

```
$ git version
git version 2.21.0.windows.1
```

If output shows Git with its version information that indicates, that Git Client installs properly.

2. To configure user level configuration of user ID and email ID execute

```
$ git config --global user.name "username"
$ git config --global user.email "username@cognizant.com"
```

3. To check if the configuration is properly set, execute the following command.

```
$ git config --global --list
user.name=username
user.email=username@cognizant.com
```

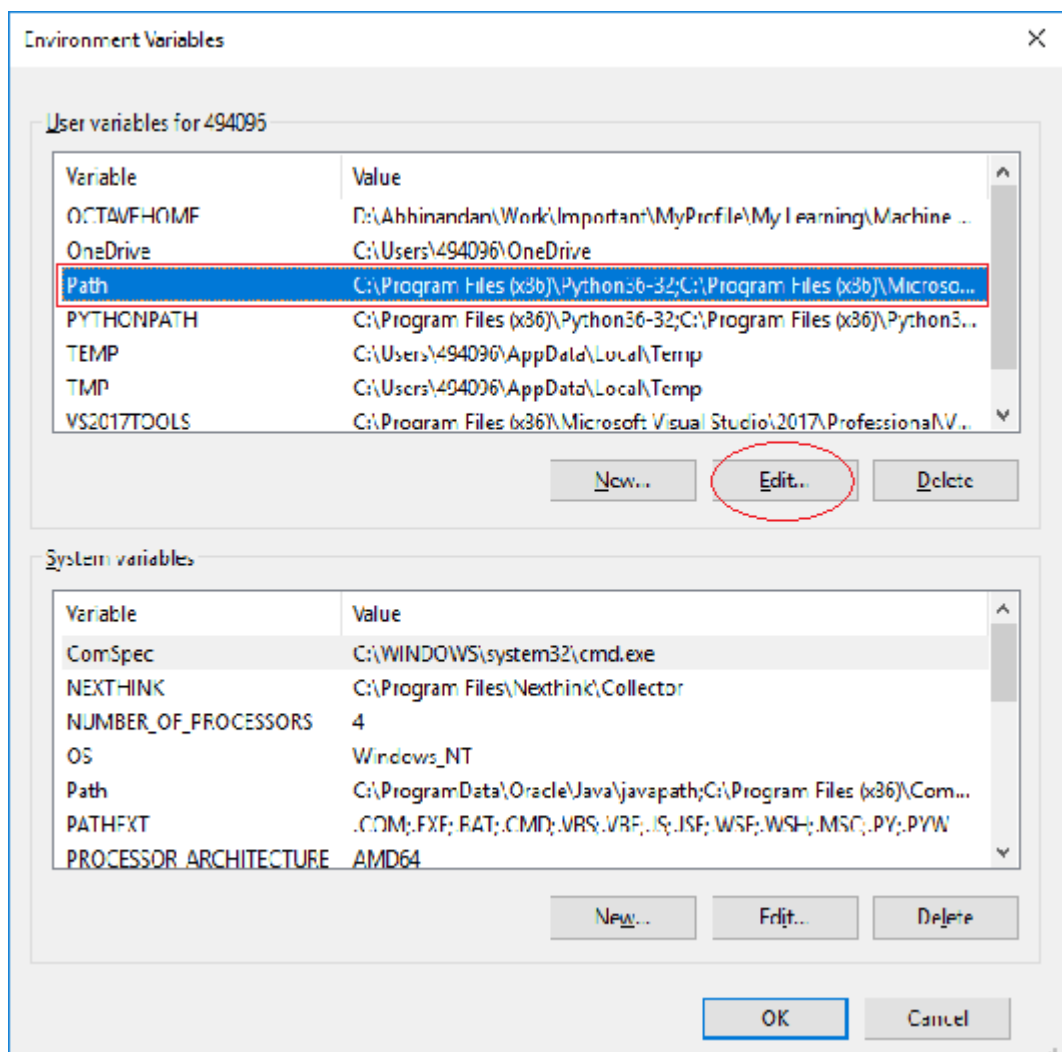
Step 2: Integrate notepad++.exe to Git and make it a default editor

1. To check, if notepad++.exe execute from Git bash

```
$ notepad++  
bash: notepad++: command not found
```

If Git bash could not able to recognize notepad++ command that implies notepad++.exe is not added to the environment path variable.

To add path of notepad++.exe to environment variable, go to control panel -> System -> Advanced System settings. Go to Advanced tab -> Environment variables -> Add path of notepad++.exe to the path user variable by clicking on "Edit"



2. Exit Git bash shell, open bash shell and execute

```
$ notepad++
```

Now, notepad++ will open from Git bash shell

3. To create an alias command for notepad++.exe, execute

```
$ notepad++.exe bash -profile
```

It will open notepad++ from bash shell, and create a user profile by adding the line in notepad++

```
alias npp='notepad++.exe -multiInst -nosession'
```

4. To configure the editor, execute the command

```
$ git config --global core.editor "notepad++.exe -multiInst -nosession"
```

5. To verify if notepad++ is the default editor, execute the command

```
$ git config --global -e  
hint: Waiting for your editor to close the file... _
```

Here '-e' option implies editor

It will show the entire global configuration as shown below,

```
[user]  
  name = username  
  email = username@cognizant.com  
[core]  
  editor = notepad++.exe -multiInst -nosession
```

Step 3: Add a file to source code repository

1. Open Git bash shell and create a new project “**GitDemo**” by executing the command

```
$ git init GitDemo  
Initialized empty Git repository in D:/Development_Avecto/GitDemo/.git/
```

2. Git bash initializes the “**GitDemo**” repository. To verify, execute the command

```
$ ls -al  
total 8  
drwxr-xr-x 1      1049089 0 Jan 13 11:54 ./  
drwxr-xr-x 1      1049089 0 Jan 13 11:54 ../  
drwxr-xr-x 1      1049089 0 Jan 13 11:54 .git/
```

It will display all the hidden files in the Git “working directory”.

3. To create a file “**welcome.txt**” and add content to the file, execute the command

```
$ echo "Welcome to the version control" >> welcome.txt
```

4. To verify if the file “welcome.txt” is created, execute

```
$ ls -al
total 9
drwxr-xr-x 1 494096 1049089  0 Jan 13 12:02 ./
drwxr-xr-x 1 494096 1049089  0 Jan 13 11:54 ../
drwxr-xr-x 1 494096 1049089  0 Jan 13 12:01 .git/
-rw-r--r-- 1 494096 1049089 31 Jan 13 12:02 welcome.txt
```

5. To verify the content, execute the command

```
$ cat welcome.txt
Welcome to the version control
```

6. Check the status by executing

```
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        welcome.txt
```

Now the file “**welcome.txt**” is available in Git “working directory”

7. To make the file to be tracked by Git repository, execute the command

```
$ git add welcome.txt
warning: LF will be replaced by CRLF in welcome.txt.
The file will have its original line endings in your working directory
```

8. To add multi line comments, we are opening default editor to comment. Execute the command

```
$ git commit
```

Notepad++ editor will open and to add multi-line comment with default editor

9. To check if local and “Working Directory” git repository are same, execute git status

```
$ git status
On branch master
nothing to commit, working tree clean
```

welcome.txt is added to the local repository.

10. Signup with GitLab and create a remote repository **“GitDemo”**

11. To pull the remote repository, execute

```
git pull origin master
```

12. To push the local to remote repository, execute

```
git push origin master
```

SOLUTION:

```
angel@Angel MINGW64 ~
$ git --version
git version 2.55.0.windows.3

$ mkdir GitDemo

angel@Angel MINGW64 ~
$ cd GitDemo

angel@Angel MINGW64 ~/GitDemo
$ git init
Initialized empty Git repository in C:/Users/angel/GitDemo/.git/

angel@Angel MINGW64 ~/GitDemo (master)
$ echo "Welcome to Git" > welcome.txt

angel@Angel MINGW64 ~/GitDemo (master)
$ cat welcome.txt
Welcome to Git

angel@Angel MINGW64 ~/GitDemo (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    welcome.txt

nothing added to commit but untracked files present (use "git add" to track)

angel@Angel MINGW64 ~/GitDemo (master)
$ git add welcome.txt
warning: in the working copy of 'welcome.txt', LF will be replaced by CRLF the next time Git touches it

angel@Angel MINGW64 ~/GitDemo (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   welcome.txt

angel@Angel MINGW64 ~/GitDemo (master)
$ git commit
hint: Waiting for your editor to close the file... error: cannot spawn notepad++: No such file or directory
error: unable to start editor 'notepad++'
Please supply the message using either -m or -F option.

angel@Angel MINGW64 ~/GitDemo (master)
$ git commit -m "Added welcome.txt"
[master (root-commit) 9871a86] Added welcome.txt
 1 file changed, 1 insertion(+)
 create mode 100644 welcome.txt

angel@Angel MINGW64 ~/GitDemo (master)
$ |
```

Objectives

- Explain git ignore
- Explain how to ignore unwanted files using git ignore

In this hands-on lab, you will learn how to:

- Implement git ignore command to ignore unwanted files and folders

Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Setting up Git environment
- Integrate notepad++ as a default editor
- A Git repository in the local system and a remote repository in GitLab

Notes*:

Please follow the below steps for creating a free account in GitHub.
Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **20 minutes.**

Create a “**.log**” file and a **log folder** in the working directory of Git. Update the **.gitignore** file in such a way that on committing, these files (.log extensions and log folders) are ignored.

Verify if the git status reflects the same about working directory, local repository and git repository.

SOLUTION:

```
angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ touch error.log

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ mkdir log

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ ls -la
total 3
drwxr-xr-x  1 angel  197609 0 May 23 11:35 .
drwxr-xr-x  1 angel  197609 0 May 23 10:20 ..
drwxr-xr-x  1 angel  197609 0 May 23 11:35 .git
-rw-r--r--  1 angel  197609 0 May 23 10:50 welcome.txt
-rw-r--r--  1 angel  197609 0 May 23 11:35 error.log
drwxr-xr-x  1 angel  197609 0 May 23 11:35 log

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    error.log
    log/

nothing added to commit but untracked files present (use "git add" to track)

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ echo "*.log" >> .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ echo "log/" >> .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ cat .gitignore
*.log
log/

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    !.gitignore

nothing added to commit but untracked files present (use "git add" to track)
```



```
angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git add .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git commit -m "Add gitignore to ignore log files and log folder"
[master 85a7b10] Add gitignore to ignore log files and log folder
1 file changed, 2 insertions(+)
create mode 100644 .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ ls -la
total 3
drwxr-xr-x  1 angel  197609 0 May 23 11:37 .
drwxr-xr-x  1 angel  197609 0 May 23 10:20 ..
drwxr-xr-x  1 angel  197609 0 May 23 11:37 .git
-rw-r--r--  1 angel  197609 18 May 23 11:37 .gitignore
-rw-r--r--  1 angel  197609 0 May 23 10:50 welcome.txt
-rw-r--r--  1 angel  197609 0 May 23 11:35 error.log
drwxr-xr-x  1 angel  197609 0 May 23 11:35 log

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ |
```

Objectives

- Explain branching and merging
- Explain about creating a branch request in GitLab
- Explain about creating a merge request in GitLab

In this hands-on lab, you will learn how to:

- Construct a branch, do some changes in the branch, and merge it with master (or trunk)

Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Setting up Git environment with P4Merge tool for Windows

Notes*:

Please follow the below steps for creating a free account in GitHub.
Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes**.

Please follow the instruction to complete the hands-on. Each instruction expects a command for the Git Bash.

Branching:

1. Create a new branch "**GitNewBranch**".
2. List all the local and remote branches available in the current trunk. Observe the "*" mark which denote the current pointing branch.
3. Switch to the newly created branch. Add some files to it with some contents.
4. Commit the changes to the branch.
5. Check the status with "**git status**" command.

Merging:

1. Switch to the master
2. List out all the differences between trunk and branch. These provide the differences in command line interface.
3. List out all the visual differences between master and branch using **P4Merge tool**.
4. Merge the source branch to the trunk.
5. Observe the logging after merging using "**git log --online --graph --decorate**"
6. Delete the branch after merging with the trunk and observe the git status.

SOLUTION:

```
angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git checkout -b GitNewBranch
Switched to a new branch 'GitNewBranch'

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitNewBranch)
$ git branch -a
* GitNewBranch
  master

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitNewBranch)
$ echo "This is a file in new branch" > branchfile.txt

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitNewBranch)
$ git add branchfile.txt

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitNewBranch)
$ git commit -m "Added branchfile in GitNewBranch"
[GitNewBranch d8f3b6c] Added branchfile in GitNewBranch
1 file changed, 1 insertion(+)
create mode 100644 branchfile.txt

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitNewBranch)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git branch -a
  GitNewBranch
* master
  remotes/origin/master
```

```
angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git diff GitNewBranch master
diff --git a/branchfile.txt b/branchfile.txt
new file mode 100644
index 0000000..d7bdbf5
--- /dev/null
+++ b/branchfile.txt
@@ -0,0 +1 @@
+This is a file in new branch

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ p4merge master GitNewBranch
Merging into target: master
Opening: C:/Program Files/Perforce/p4merge/p4merge.exe

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git merge GitNewBranch
Updating 9871a86..d8f3b6c
Fast-forward
 branchfile.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 branchfile.txt

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate
↑ d8f3b6c (HEAD -> master, GitNewBranch) Added branchfile in GitNewBranch
* 9871a86 Added welcome.txt

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was d8f3b6c).

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```

Objectives

- Explain how to resolve the conflict during merge.

In this hands-on lab, you will learn how to:

- Implement conflict resolution when multiple users are updating the trunk (or master) in such a way that it results into a conflict with the branch's modification.

Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Hands-on ID: **"Git-T03-HOL_001"**

Notes*:

Please follow the below steps for creating a free account in GitHub.
Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes**.

Please follow the instructions to complete the hands-on. Each instruction expect a command for the Git Bash.

1. Verify if master is in clean state.
2. Create a branch **"GitWork"**. Add a file "hello.xml".
3. Update the content of "hello.xml" and observe the status
4. Commit the changes to reflect in the branch
5. Switch to master.
6. Add a file **"hello.xml"** to the master and add some different content than previous.
7. Commit the changes to the master
8. Observe the log by executing **"git log --oneline --graph --decorate --all"**
9. Check the differences with Git diff tool
10. For better visualization, use P4Merge tool to list out all the differences between master and branch
11. Merge the bran to the master
12. Observe the git mark up.
13. Use 3-way merge tool to resolve the conflict
14. Commit the changes to the master, once done with conflict
15. Observe the git status and add backup file to the .gitignore file.
16. Commit the changes to the .gitignore
17. List out all the available branches
18. Delete the branch, which merge to master.
19. Observe the log by executing **"git log --oneline --graph --decorate"**

SOLUTION:

```
angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitWork)
$ echo "<greeting>Hello from GitWork branch</greeting>" > hello.xml

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitWork)
$ git status
On branch GitWork
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.xml

nothing added to commit but untracked files present (use "git add" to track)

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitWork)
$ git add hello.xml

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitWork)
$ git commit -m "Add hello.xml in GitWork branch"
[GitWork 3a1f9b8] Add hello.xml in GitWork branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (GitWork)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ echo "<greeting>Hello from MASTER branch</greeting>" > hello.xml

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git add hello.xml

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git commit -m "Add hello.xml in mastok branch"
[master 7d6e5a1] Add hello.xml in master branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml
```



```

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate --all
* 7d6e5a1 (HEAD -> master) Add hello.xml in master branch
| * 3a1f9b8 (GitWork) Add hello.xml in Gitwork branch
|/
* 9871a86 Add welcome.txt
* 4d7ele3 Add gitignore to ignore log files and log folder

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git diff GitWork master
diff --git a/hello.xml b/hello.xml
index e69de29..506ab6c 100644
--- a/hello.xml
+++ b/hello.xml
@@ -0,0 +1 @@
+<greeting>Hello from MASTER branch</greeting>

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ p4merge master GitWork
Merging into: master
Opening: C:/Program Files/Perforce/p4merge/p4merge.exe

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git merge GitWork
Auto-merging hello.xml
CONFLICT (add/add): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both added:      hello.xml

no changes added to commit (use "git add" and/or "git commit -a")

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master|MERGING)
$ p4merge
Merging: hello.xml

```

```

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master|MERGING)
$ git add hello.xml

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master|MERGING)
$ git commit -m "Resolve merge conflict and merge GitWork into master"
[master blf2c3d] Resolve merge conflict and merge GitWork into master

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ echo "hello.xml~" >> .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git add .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git commit -m "Add backup file hello.xml~ to .gitignore"
[master 0a9d4e5] Add backup file hello.xml~ to .gitignore
1 file changed, 1 insertion(+)
create mode 100644 .gitignore

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git branch -a
  GitWork
* master
  remotes/origin/master

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git branch -d GitWork
Deleted branch GitWork (was 3a1f9b8).

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate --all
* 0a9d4e5 (HEAD -> master) Add backup file hello.xml~ to .gitignore
* blf2c3d Resolve merge conflict and merge GitWork into master
* 7d6e5a1 Add hello.xml in master branch
* 9871a86 Add welcome.txt
* 4d7e1e3 Add gitignore to ignore log files and log folder

```


Objectives

- Explain how to clean up and push back to remote Git

In this hands-on lab, you will learn how to:

- Execute steps involving clean up and push back to remote Git.

Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Hands-on ID: “**Git-T03-HOL_002**”

Notes*:

Please follow the below steps for creating a free account in GitHub.
Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **10 minutes.**

Please follow the instructions to complete the hands-on. Each instruction expects a command for the Git Bash.

20. Verify if master is in clean state.
21. List out all the available branches.
22. Pull the remote git repository to the master
23. Push the changes, which are pending from “**Git-T03-HOL_002**” to the remote repository.
24. Observe if the changes are reflected in the remote repository.

SOLUTION:

```
angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git branch
  GitWork
* master

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git pull origin master
From https://github.com/angelferosha/GitDemo
 * branch                master       -> FETCH_HEAD
Already up to date.

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git push origin master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 328 bytes | 328.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0)
To https://github.com/angelferosha/GitDemo.git
  3a1f9b8..7d6e5a1  master -> master

angel@LAPTOP-8JH2K33 MINGW64 ~/GitDemo (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```